Unit I : Assembler

Course : Language Processor and Compiler Construction

Manisha Mali
manisha.mali@viit.ac.in

BRACT'S, Vishwakarma Institute of Information Technology, Pune-48
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
(NBA and NAAC accredited, ISO 9001:2015 certified)

Department of Computer Engineering

Content

Part – I : Introduction

- Need of System Software,
- Components of System Software
- Language Processing Activities
- Fundamentals of Language Processing
- Interpreter

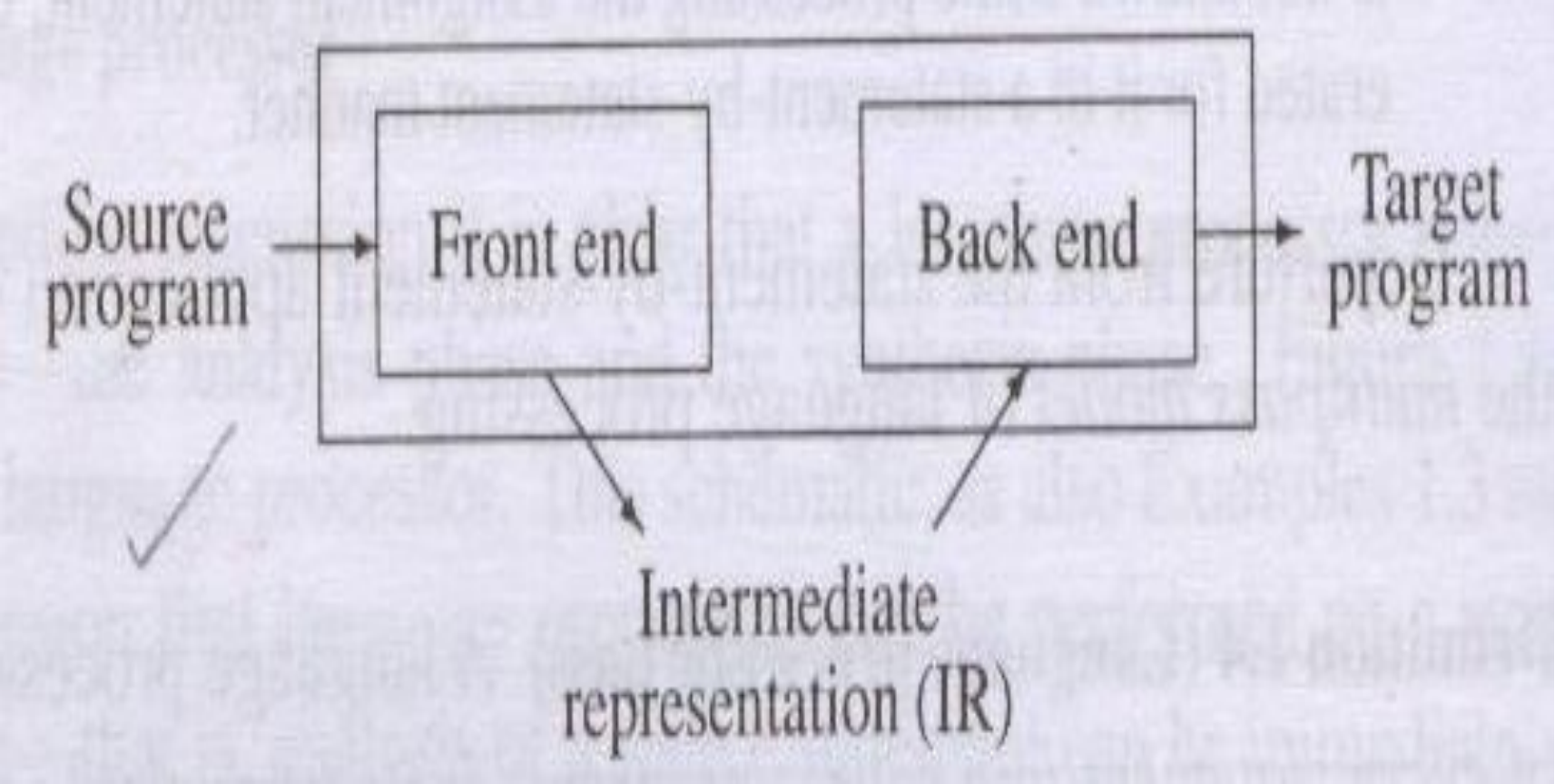
Part- II : Assemblers

- **Elements of Assembly Language Programming**
- **A simple Assembly Scheme**
- **Pass structure of Assemblers**
- **Design of Two Pass Assembler**

Passes of a language processor

- Pass I
  - Performs analysis of the source program and note relevant information
- Pass II
  - Perform synthesis of target program

Intermediate representation



ASSEMBLER

- Assembler is a program that turns symbols into machine instructions.
- Assembler is a program which translates assembly language program in machine language program.

Introduction to assembly language

- Mnemonic operation code (Mnemonic opcode)
- Symbolic operands
- Data declarations

```
        START   101
        READ    N                   101) + 09 0 113
        MOVER   BREG, ONE           102) + 04 2 115
        MOVEM   BREG, TERM          103) + 05 2 116
AGAIN   MULT    BREG, TERM          104) + 03 2 116
        MOVER   CREG, TERM          105) + 04 3 116
        ADD     CREG, ONE           106) + 01 3 115
        MOVEM   CREG, TERM          107) + 05 3 116
        COMP    CREG, N             108) + 06 3 113
        BC      LE,   AGAIN         109) + 07 2 104
        MOVEM   BREG, RESULT        110) + 05 2 114
        PRINT   RESUL               111) + 10 0 114
                T
        STOP                        112) + 00 0 000
N       DS      1                   113)
RESULT  DS      1                   114)
ONE     DC      '1'                 115)
TERM    DS      1                   116)
        END
```

Assembly Language Syntax

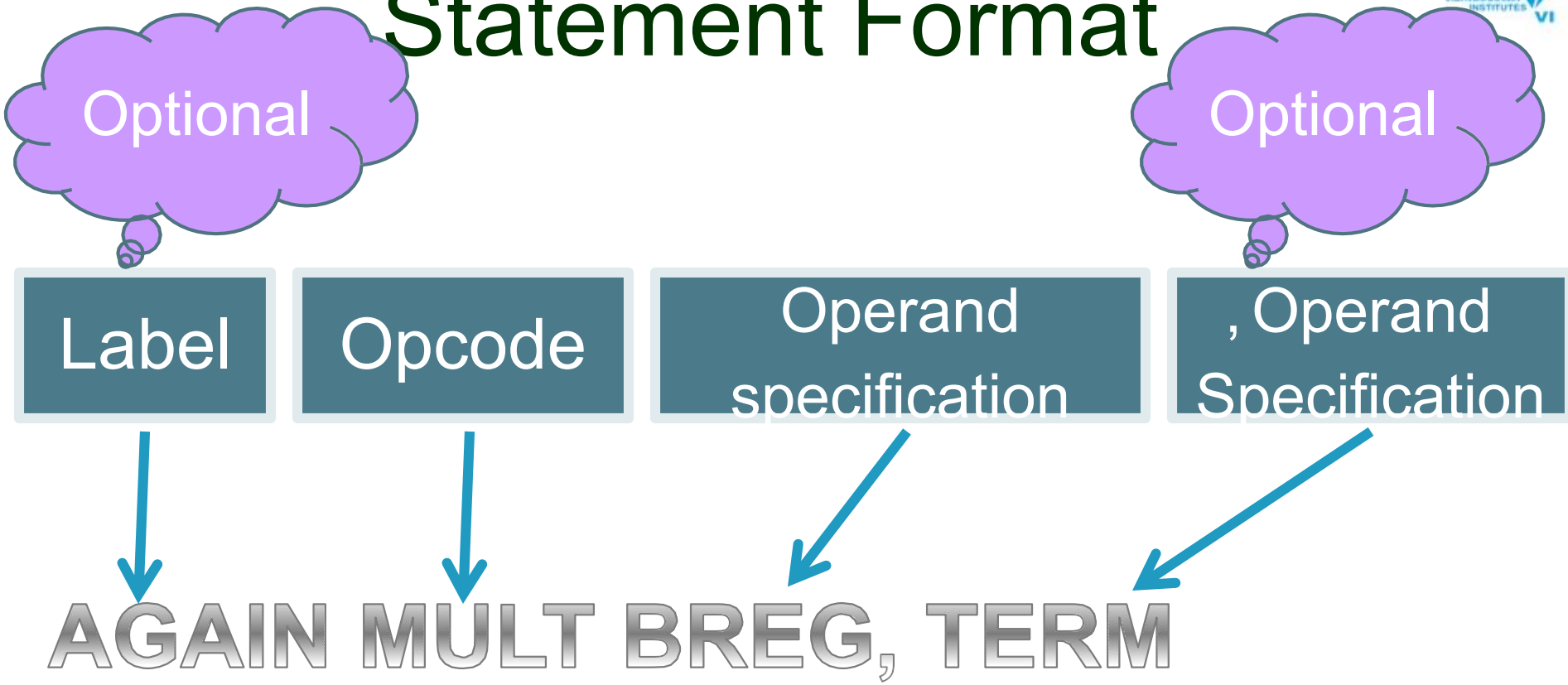
- Each line of a program is one of the following:
  - an instruction
  - an assember directive (or pseudo-op)
  - a comment
- Whitespace (between symbols) and case are ignored.
- Comments (beginning with “;”) are also ignored.

- An instruction has the following format:

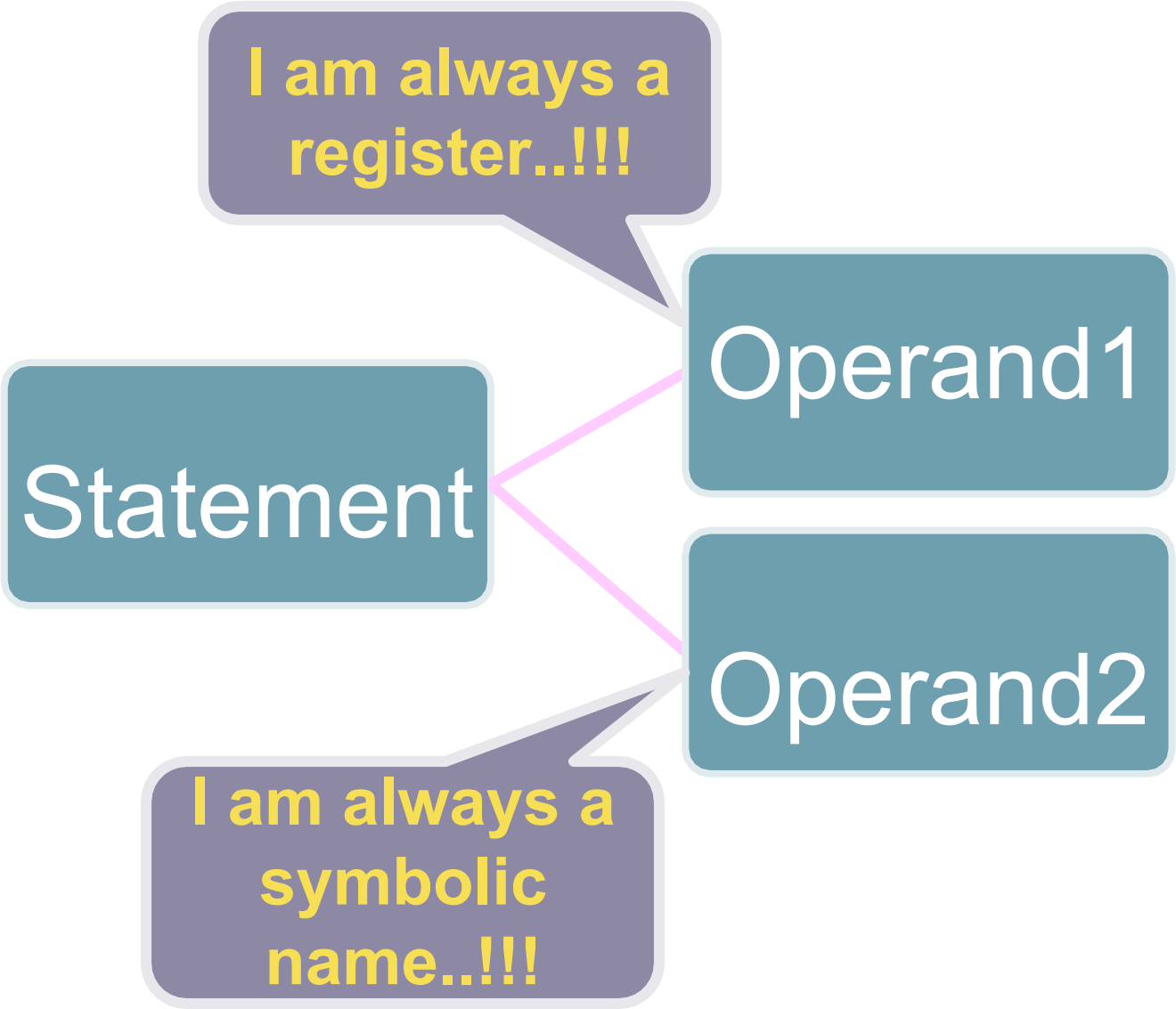
LABEL OPCODE OPERANDS ; COMMENTS

optional *mandatory*

Statement Format

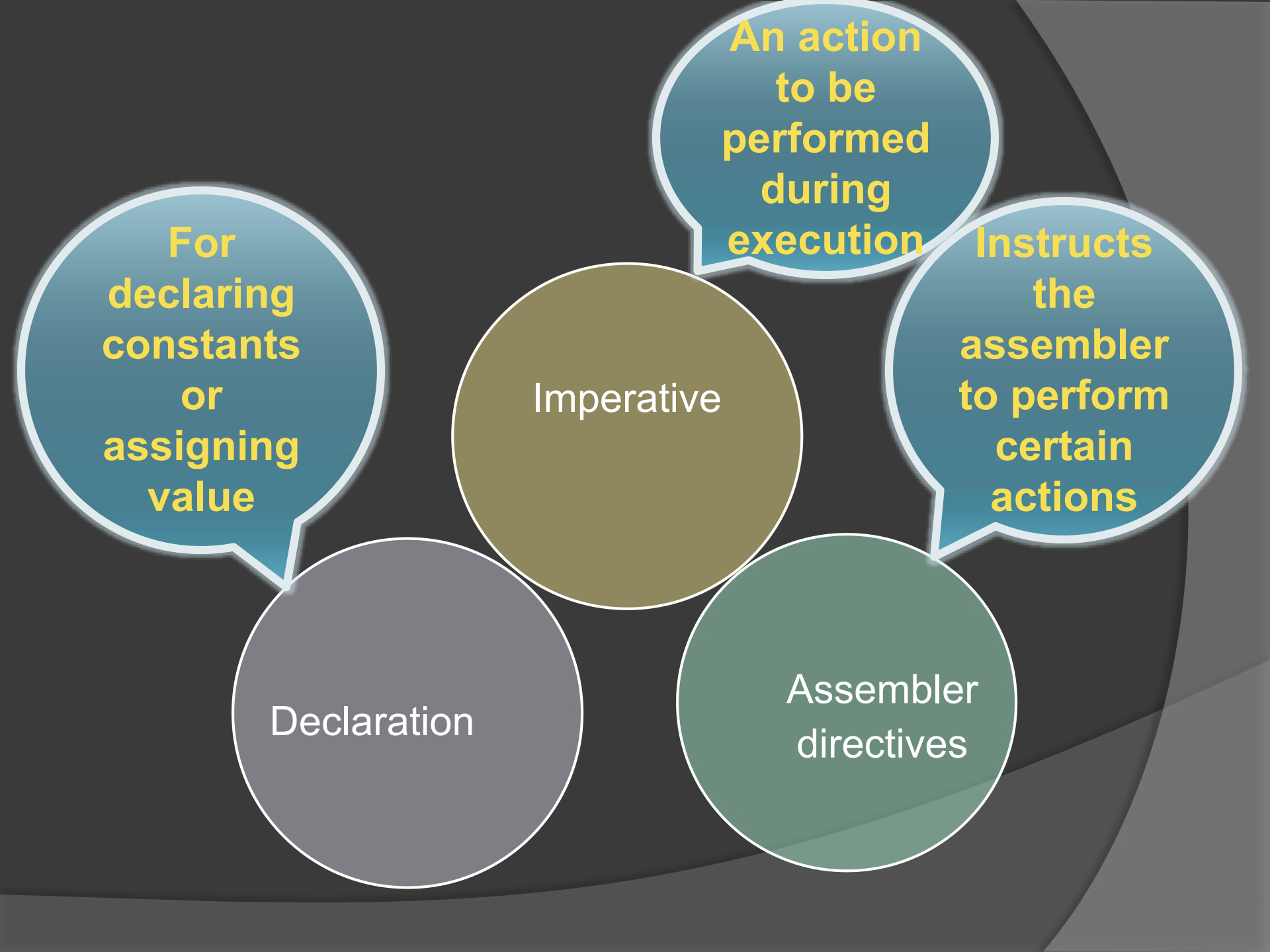


A simple assembly language



Assembly language statements

- Imperative statements
  - Indicate an action to be performed during the execution.
- Declarative statements
  - [label] DS <constant>
  - [label] DC <value>
    - A DS 3
    - B DS 1
    - ONE DC ‘1’
- Assembler directives
  - START <constant>
  - END [<operand spec >]


For declaring constants or assigning value
Declaration
Imperative
An action to be performed during execution
Instructs the assembler to perform certain actions
Assembler directives

Declaration Statements

DS

(Declare Storage)

- Reserves area of memory and associates names with them
- Example : A DS 1

DC

(Declare constant)

- Construct memory words containing constants
- ONE DC '1'

1. A constant as immediate operand

```
ADD    AREG,5
```

Our assembly language doesn't support this..!!

How to write equivalent instructions for this??

```
                ADD
                AREG,FIVE
FIVE              ----
```

2. Literal

???
??

What is the Difference between literal and immediate operand ??

(Literal)

ADD AREG, ='5'

(Imm. operand)

ADD AREG,5

No Architectural provision is needed for literal like immediate operand..

Assembler Directives

1. START <constant>

The first word of the target program should be placed in the memory word with the address specified..

Indicates the end of the source program..

2. END [<op spec>]

An assembly language program for factorial of a number (N!).

```
        START   101
        READ    N
        MOVER   BREG, ONE
        MOVEM   BREG, TERM
AGAIN   MULT    BREG, TERM
        MOVER   CREG, TERM
        ADD     CREG, ONE
        MOVEM   CREG, TERM
        COMP    CREG, N
        BC      LE, AGAIN
        MOVEM   BREG, RESULT

        PRINT   RESULT
        STOP
N       DS      1
RESULT  DS      1
ONE     DC      '1'
TERM    DS      1
        END
```

Advantages of assembly language

- Machine language program needs to be changed drastically if we modify the original program.

- It is more suitable when it is desirable to use specific architectural features of a computer…
- Example – special instructions supported by CPU

	START	101	
	READ	N	
	MOVER	BREG, ONE	
	MOVEM	BREG, TERM	
AGAIN	MULT	BREG, TERM	
	MOVER	CREG, TERM	
	ADD	CREG, ONE	
	MOVEM	CREG, TERM	
	COMP	CREG, N	
	BC	LE, AGAIN	
	DIV	BREG, TWO	
	MOVEM	BREG, RESULT	
	PRINT	RESULT	
	STOP		
N	DS	1	
RESUL	DS	1	
T ONE	DC	'1'	
TERM	DS	1	
TWO	DC	'2'	
	END		

101) + 09 0
114
102) + 04 2
116
103) + 05 2
117
104) + 03 2
117
105) + 04 3
117
106) + 01 3
111)116 + 05 0
107)115 + 05 3
112)117 + 10 0
108)115 + 06 3
113)114 + 00 0
109)000 + 07 2
114)104
116) + 08 2
118
116)
117)

Data structure to implement Memory allocation

initialized to constant in **START**

Location Counter

Contains the **address of the next mem. word**

Whenever there is a label, it enters the **Label** and **LC contents** in the new entry of **symbol table**

Name	Address
AGAIN	104

Cont…

After this, it finds the **number of mem words required by that statement** and again updates the LC content

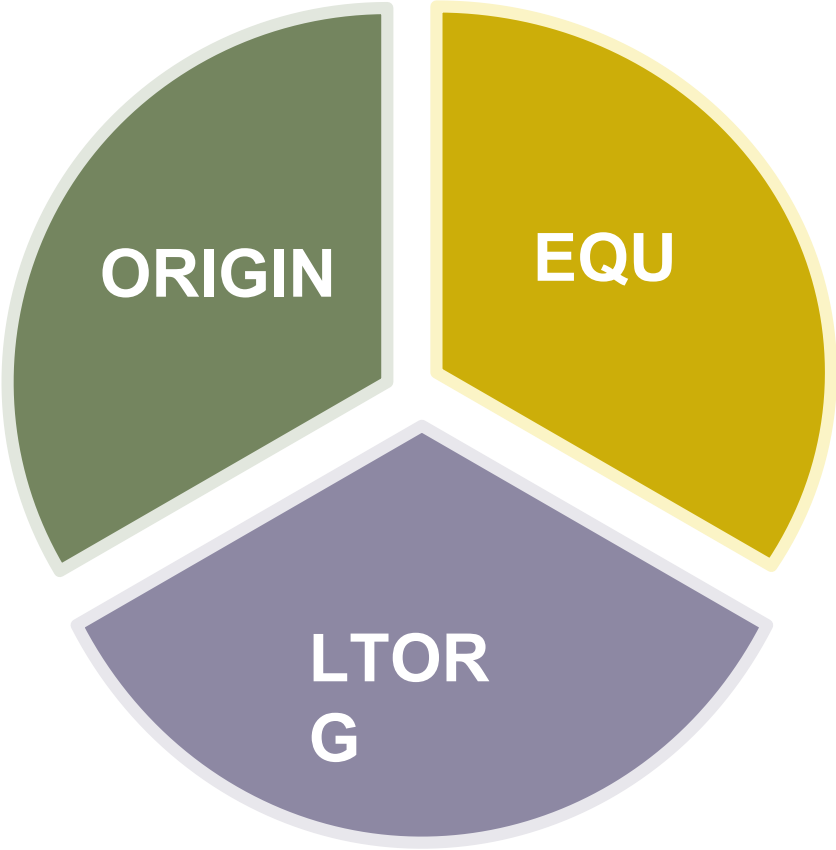
It needs to know the **length of instructions.!**

Depends on assembly lang.

Processing involved in maintaining LC

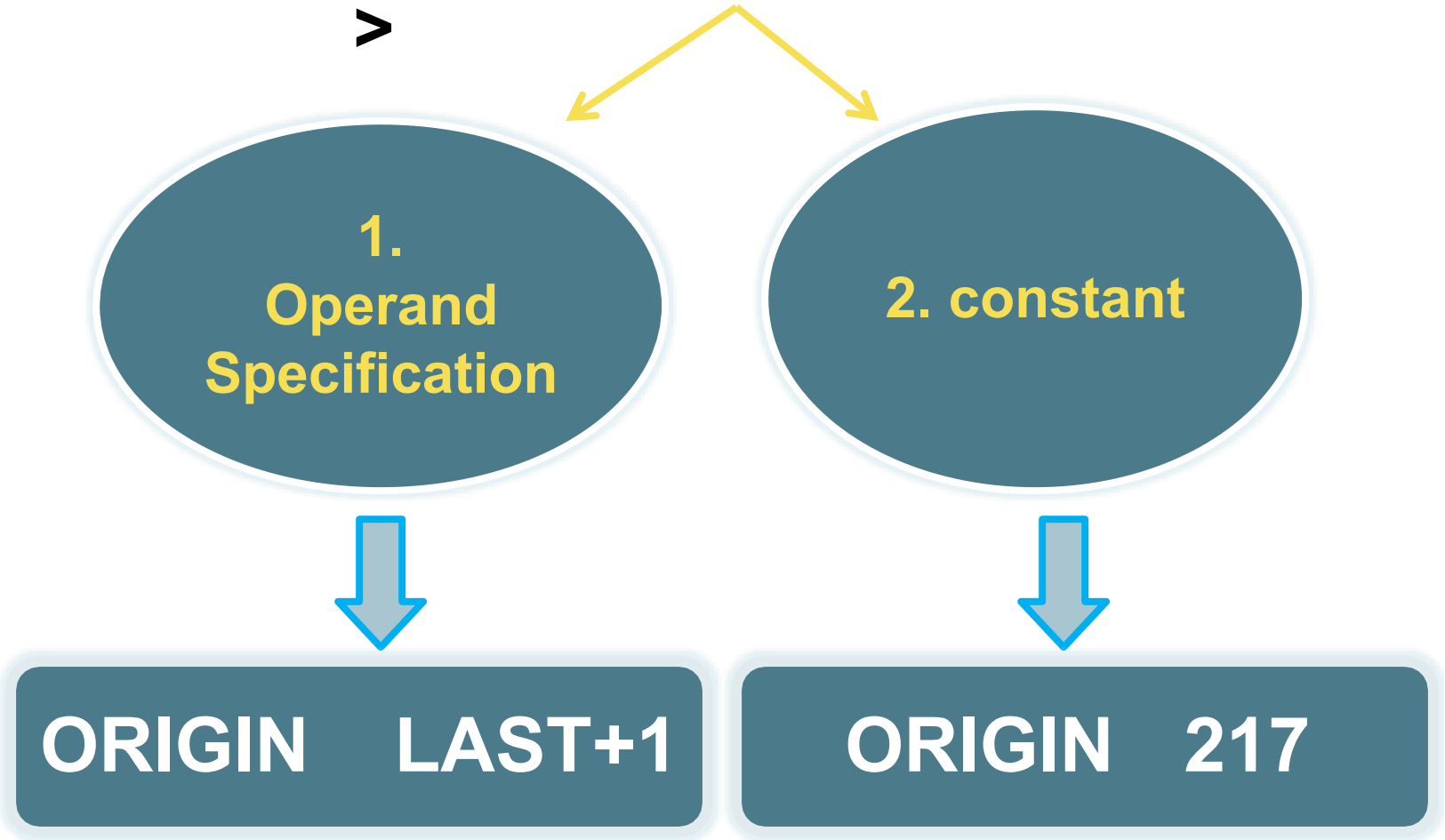
LC Processing

Advanced Assembler Directives



ORIGIN

- **ORIGIN < Address Specification >**



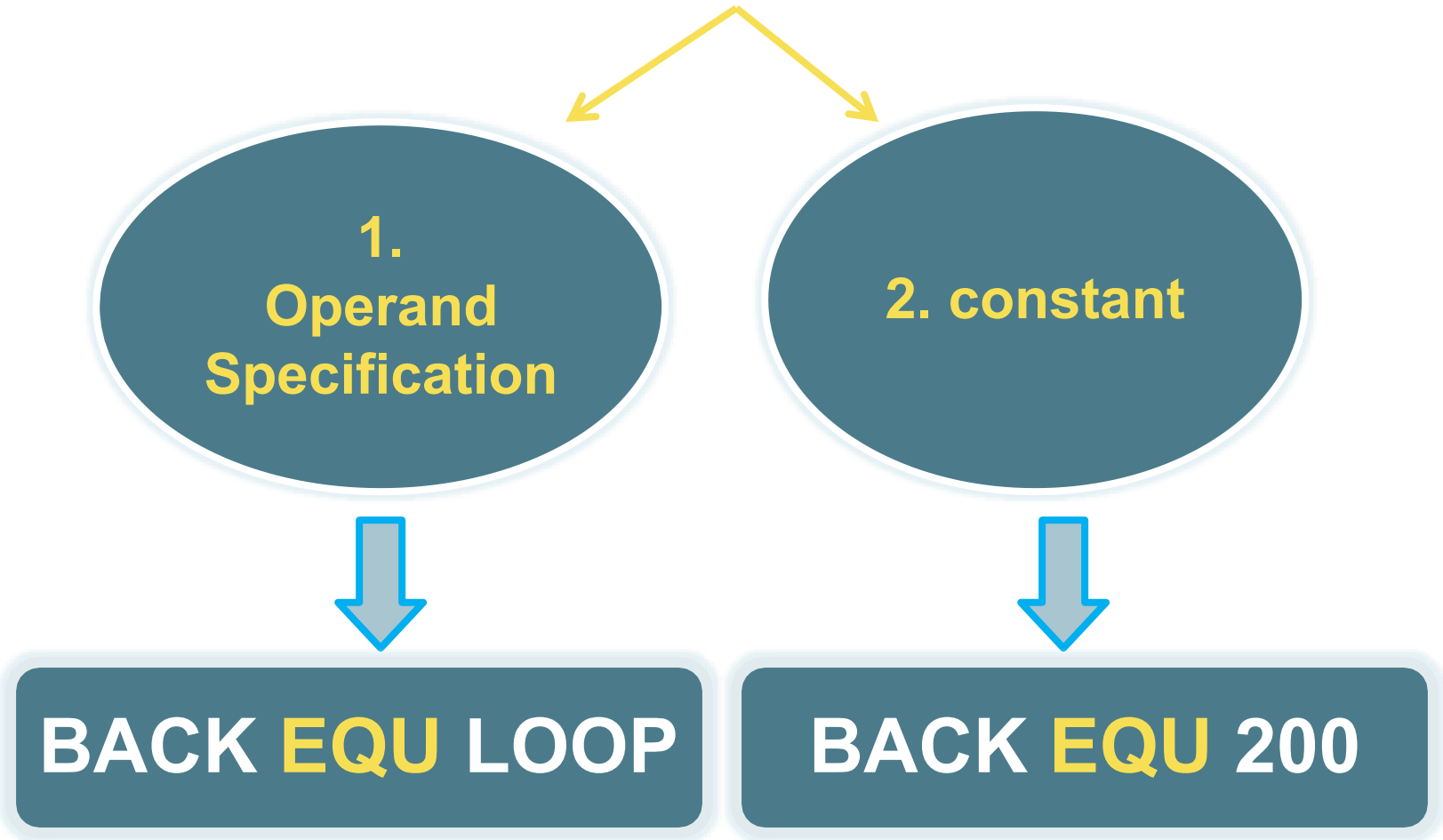
Cont…

- It is useful when your target program does not consist of **consecutive** memory words.
- Operand Specification – Ability to perform
  Relative LC Processing, not absolute.
- Difference between using both the options

```
1          START   200
2          MOVER   AREG, ='5'        200)  +04 1 211
3          MOVEM   AREG, A           201)  +05 1 217
4   LOOP   MOVER   AREG, A           202)  +04 1 217
5          MOVER   CREG, B           203)  +05 3 218
6          ADD     CREG, ='1'        204)  +01 3 212
7          ...

12         BC      ANY, NEXT         210)  +07 6 214
13         LTORG
                   ='5'              211)  +00 0 005
                   ='1'              212)  +00 0 001
14         ...
15  NEXT   SUB     AREG, ='1'        214)  +02 1 219
16         BC      LT, BACK          215)  +07 1 202
17  LAST   STOP                      216)  +00 0 000
18         ORIGIN  LOOP+2
19         MULT    CREG, B           204)  +03 3 218
20         ORIGIN  LAST+1
21  A      DS      1                 217)
22  BACK   EQU     LOOP
23  B      DS      1                 218)
24         END
25                 ='1'              219)  +00 0 001
```

EQU

- Defines the symbol to represent <add spec>
- **Symbol EQU < address specification >**



```
 1          START   200
 2          MOVER   AREG, ='5'        200)  +04 1 211
 3          MOVEM   AREG, A           201)  +05 1 217
 4   LOOP   MOVER   AREG, A           202)  +04 1 217
 5          MOVER   CREG, B           203)  +05 3 218
 6          ADD     CREG, ='1'        204)  +01 3 212
 7          ...

12          BC      ANY, NEXT         210)  +07 6 214
13          LTORG
                    ='5'              211)  +00 0 005
                    ='1'              212)  +00 0 001
14          ...
15   NEXT   SUB     AREG, ='1'        214)  +02 1 219
16          BC      LT, BACK          215)  +07 1 202
17   LAST   STOP                      216)  +00 0 000
18          ORIGIN  LOOP+2
19          MULT    CREG, B           204)  +03 3 218
20          ORIGIN  LAST+1
21   A      DS      1                 217)
22   BACK   EQU     LOOP
23   B      DS      1                 218)
24          END
25                  ='1'              219)  +00 0 001
```

Literal, why LTORG?

```
ADD    AREG, =5
```

What is done internally by assembler?

LTORG

- Allows **programmer** to **specify** where literals should be stored.

```
 1          START   200
 2          MOVER   AREG, ='5'        200)  +04 1 211
 3          MOVEM   AREG, A           201)  +05 1 217
 4   LOOP   MOVER   AREG, A           202)  +04 1 217
 5          MOVER   CREG, B           203)  +05 3 218
 6          ADD     CREG, ='1'        204)  +01 3 212
 7          ...

12          BC      ANY, NEXT         210)  +07 6 214
13          LTORG
                    ='5'              211)  +00 0 005
                    ='1'              212)  +00 0 001
```

What if we don't write LTORG?

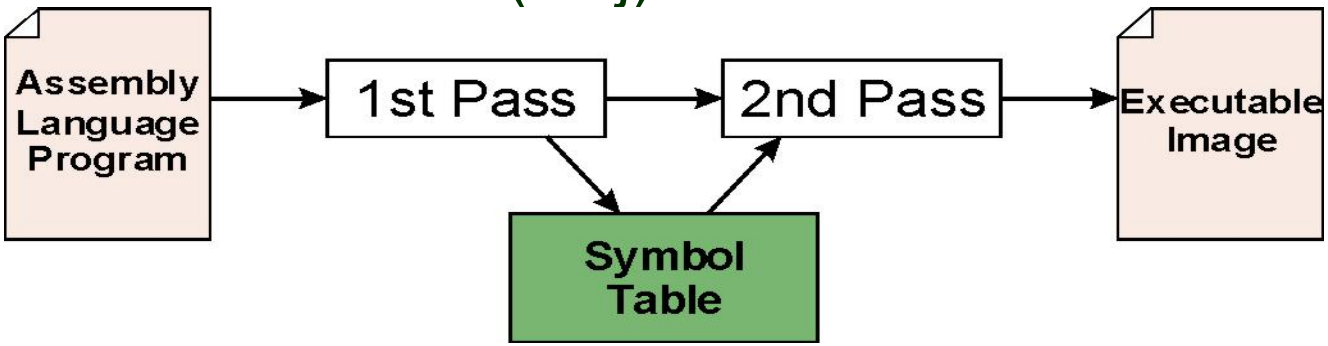
```
 1          START   200
 2          MOVER   AREG, ='5'        200)  +04 1 211
 3          MOVEM   AREG, A           201)  +05 1 217
 4   LOOP   MOVER   AREG, A           202)  +04 1 217
 5          MOVER   CREG, B           203)  +05 3 218
 6          ADD     CREG, ='1'        204)  +01 3 212
 7          ...

12          BC      ANY, NEXT         210)  +07 6 214
13          LTORG
                    ='5'              211)  +00 0 005
                    ='1'              212)  +00 0 001
14          ...
15   NEXT   SUB     AREG, ='1'        214)  +02 1 219
16          BC      LT, BACK          215)  +07 1 202
17   LAST   STOP                      216)  +00 0 000
18          ORIGIN  LOOP+2
19          MULT    CREG, B           204)  +03 3 218
20          ORIGIN  LAST+1
21   A      DS      1                 217)
22   BACK   EQU     LOOP
23   B      DS      1                 218)
24          END
25                  ='1'              219)  +00 0 001
```

Design Specification of assembler

- Identify the information necessary to perform the task
- Design the suitable data structures to record the information
- Determine the processing necessary to obtain and manage the information
- Determine the information necessary to perform the task

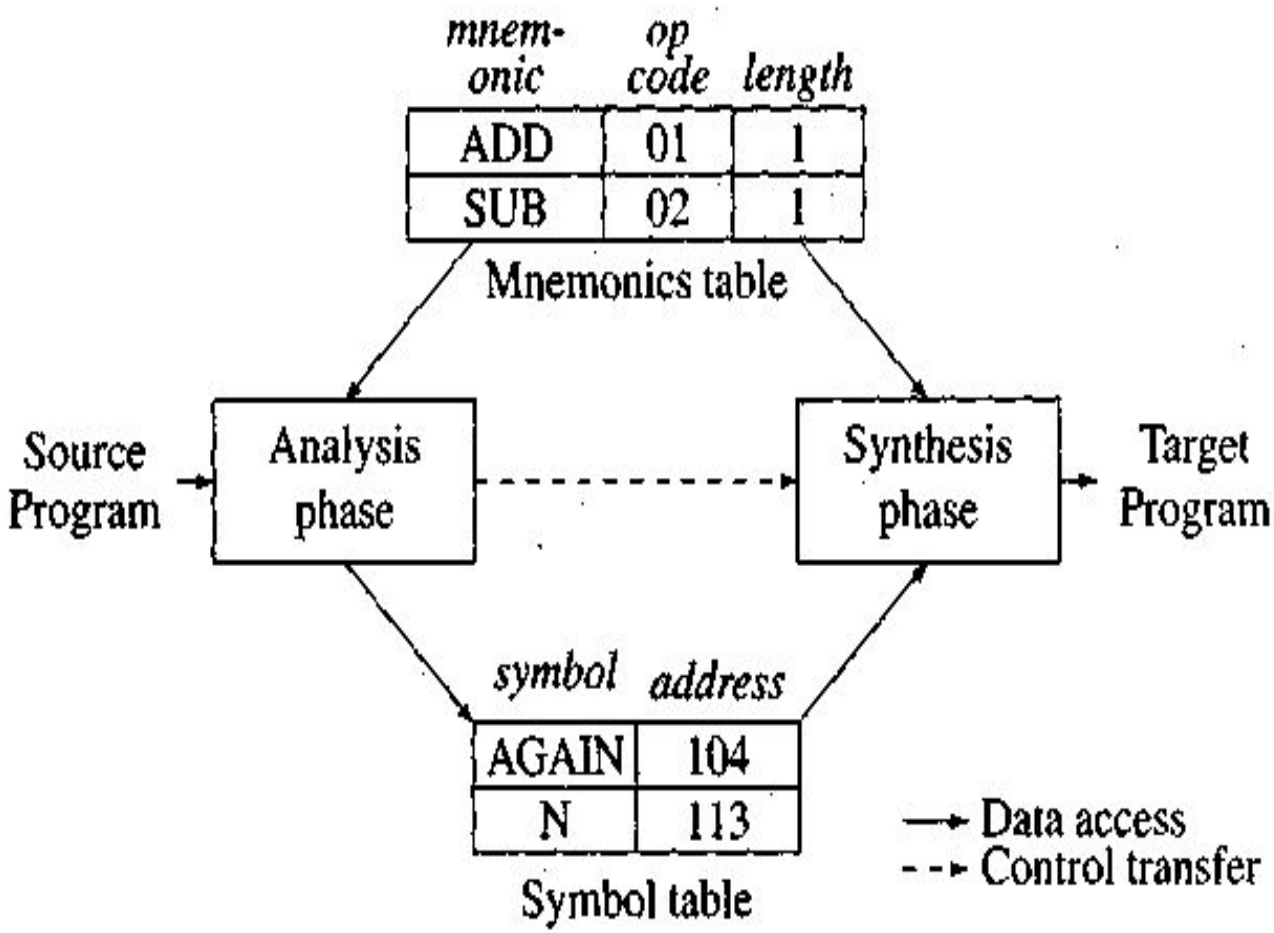
Assembly Process

- Convert assembly language file (.asm) into an executable file (.obj) .



- First Pass:
  - scan program file
  - find all labels and calculate the corresponding addresses;
    this is called the *symbol table*
- Second Pass:
  - convert instructions to machine language, using information from symbol table

Data structures of the assembler



Data structures for two pass assembler

- Mnemonic opcode table
- Symbol table
- Literal table
- Pool table

Mnemonic Operation Codes (Mnemonic opcode)

Instruction Opcode	Assembly Mnemonic	Remarks
00	STOP	Stops execution
01	ADD	First operand is
02	SUB	Modified
03	MULT	
04	MOVER	Register memory move
05	MOVEM	Memory register move
06	COMP	Sets condition code
07	BC	Branch on Condition
08	DIV	Analogous to SUB
09	READ	Performs reading and
10	PRINT	printing

Code for declaration statements and directives

Declaration statements	
DC	01
DS	02

Assembler directives	
START	01
END	02
ORIGIN	03
EQU	04
LTORG	05

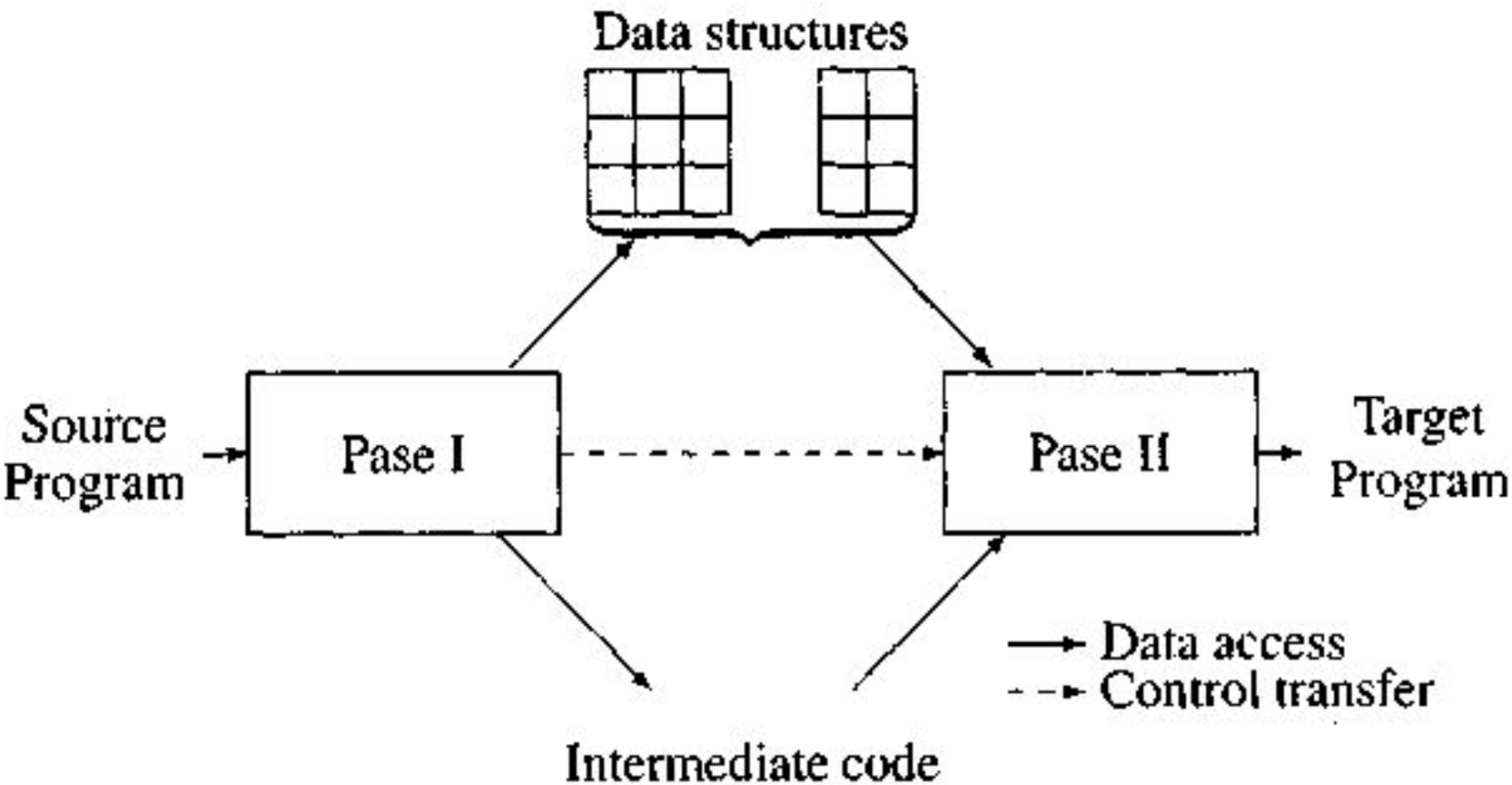
Syntax for BC

BC	Condition code	Memory Address

LT, LE, EQ, GT, GE, ANY

What if we want to have an unconditional jump?

Overview of two pass assembler



Tasks of two pass assembler

- Pass 1
  - Separate the symbol, mnemonic opcode & operand fields.
  - Build the symbol table, Literal table, Pool table
  - Perform LC processing
  - Construct intermediate representation.
- Pass 2
  - Synthesize the target program.
  - Evaluate fields and generate code
  - Process pseudo –opcodes

Data structures of assembler pass I

mnemonic opcode	class	mnemonic info
MOVER	IS	(04,1)
DS	DL	R#7
START	AD	R#11
	:	

OPTAB

symbol	address	length
LOOP	202	1
NEXT	214	1
LAST	216	1
A	217	1
BACK	202	1
B	218	1

SYMTAB

	literal	address
1	='5'	
2	='1'	
3	='1'	

LITTAB

literal no
#1
#3
–

POOLTAB

POOLTAB

```
1          START   200
2          MOVER   AREG, ='5'        200)  +04 1 211
3          MOVEM   AREG, A           201)  +05 1 217
4   LOOP   MOVER   AREG, A           202)  +04 1 217
5          MOVER   CREG, B           203)  +05 3 218
6          ADD     CREG, ='1'        204)  +01 3 212
7          ...

12         BC      ANY, NEXT         210)  +07 6 214
13         LTORG
                   ='5'              211)  +00 0 005
                   ='1'              212)  +00 0 001
14         ...
15  NEXT   SUB     AREG, ='1'        214)  +02 1 219
16         BC      LT, BACK          215)  +07 1 202
17  LAST   STOP                      216)  +00 0 000
18         ORIGIN  LOOP+2
19         MULT    CREG, B           204)  +03 3 218
20         ORIGIN  LAST+1
21  A      DS      1                 217)
22  BACK   EQU     LOOP
23  B      DS      1                 218)
24         END
25                 ='1'              219)  +00 0 001
```

	literal	address
1	='5'	
2	='1'	
3	='1'	

LITTAB

	first	# literals
1	1	2
2	3	1
3	4	0

POOLTAB

For the following program which is fed to assembler, show the contents of symbol table and literal table at the end of pass-1. (Assume size of instruction is 1)

```
        START   100
A       DS      1
L1      LOAD    B
        ADD     C
        STORE   D
D       EQU     A+1
L2      PRINT   D
        ORIGIN  A+2
C       DC      5
        ORIGIN  L2+1
        STOP
B       DC      '19'
        END
```

For the following program which is fed to assembler, show the contents of symbol table and literal table at the end of pass-1. (Assume size of instruction is 1)

	START	100	LC
A	DS	1	100
L1	LOAD	B	101
	ADD	C	102
	STORE	D	103
D	EQU	A+1	
L2	PRINT	D	104
	ORIGIN	A+2	
C	DC	5	102
	ORIGIN	L2+1	
	STOP		105
B	DC	'19'	106
	END		

Symbol Table

SYMBOL NAME	ADDRESS	LENGTH
A	100	1
L1	101	1
B	106	1
C	102	1
D	101	1
L2	104	1

Ex : For the following piece of assembly code, show the contents of symbol table, literal table, pool table and intermediate code after pass I. (8 Mks)

```
        START 100
A       DC  01
        LOAD   A
        ADD   ='5'
        MULT   ='10'
        TRANS   L
        LTORG
              ='5'
              ='10'
L       ADD   ='5'
        ADD   B
B       DS   1
        END
```

Ex : For the following piece of assembly code, show the contents of symbol table, literal table, pool table and intermediate code after pass I. (8 Mks)

```
        START 100
A       DC  01
        LOAD   A
        ADD   ='5'
        MULT   ='10'
        TRANS   L
L       ADD   '=5'
        ADD   B
B       DS   1
        END
```

Ex : For the following piece of assembly code, show the contents of symbol table, literal table, pool table and intermediate code after pass I. (8 Mks)

```
        START 100
A       DC  01
        LOAD   A
        ADD   ='6'
        MULT   ='18'
        LTORG
        TRANS   L
L       ADD   ='5'
        ADD   B
        ADD ='4'
        LTORG
M       ADD   ='5'
        ADD   ='9'
B       DS   1
        END
```

Ex : For the following piece of assembly code, show the contents of symbol table, literal table, pool table and intermediate code after pass I. (8 Mks)

```
        START 400
A       DS   05
        LOAD   A
        ADD   ='6'
D       EQU A
        MULT   ='18'
        LTORG
        ADD F
L       ADD   ='5'
        ORIGIN D+100
        ADD   B
        ADD ='4'
        LTORG
M       ADD   ='5'
        ORIGIN L+3
        AD   ='9'
        SUB ='6'
B       DS   3
A       DC  2
        END
```

Disclaimer : Instructions in Program are written randomly, will not match with any logic.

```
        START   200
        MOVER   AREG,='5'
        MOVEM   AREG,X
L1      MOVER   BREG,='2'
        ORIGIN  L1+3
        LTORG
NEXT    ADD     AREG,='1'
        SUB     BREG,='2'
        BC      LT, BACK
        LTORG
BACK    EQU     L1
        ORIGIN  NEXT+9
        MULT    CREG, X
        STOP
X       DS      1
        END
```

Ex : For the following piece of assembly code, show the contents of symbol table, literal table, pool table

```
        START   200             LC =0
        MOVER   AREG,='5'       200
        MOVEM   AREG,X          201
L1      MOVER   BREG,='2'       202
        ORIGIN  L1+3            LC=205
        LTORG
NEXT    ADD     AREG,='1'       207
        SUB     BREG,='2'       208
        BC      LT, BACK        209
        LTORG
BACK    EQU     L1
        ORIGIN  NEXT+9          LC=216
        MULT    CREG, X         216
        STOP                    217
X       DS      1               218
        END
```

SYMBOL TABLE (ST)

	SYMBOL	ADDRESS
0	L1	202
1	NEXT	207
2	BACK	202
3	X	218

LITERAL TABLE (LT)

	VALUE	ADDRESS
0	='5'	205
1	='2'	206
2	='1'	210
3	='2'	211

	POOL TABLE
0	0
1	2

An Intermediate code unit

Address	Opcode	Operands

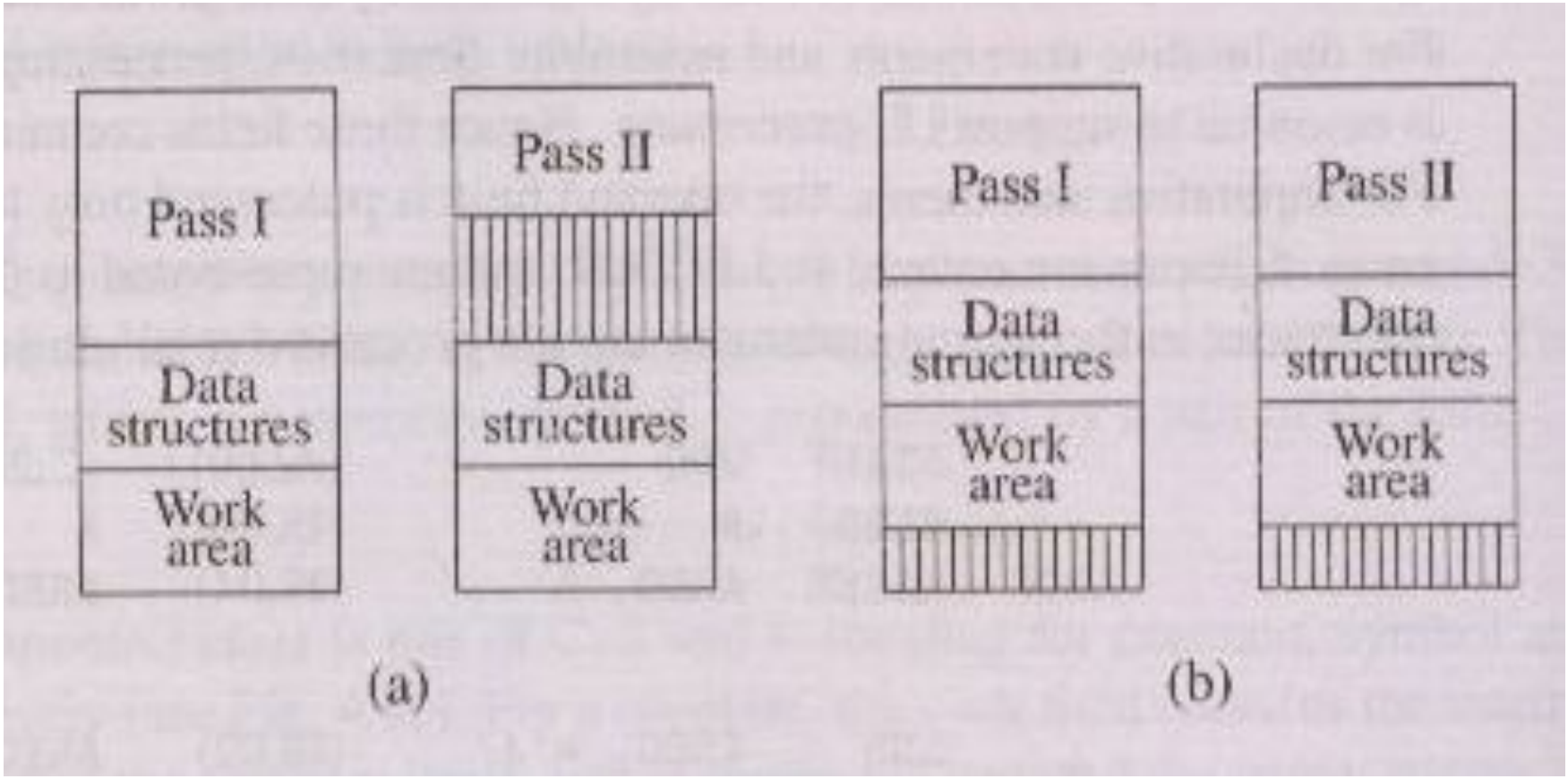
Intermediate code – variant I

```
        START   200             (AD,01)   (C,200)
        READ    A               (IS,09)   (S,01)
LOOP    MOVER   AREG, A         (IS,04)   (1)(S,01)
          :                       :
        SUB     AREG, ='1'      (IS,02)   (1)(L,01)
        BC      GT, LOOP        (IS,07)   (4)(S,02)
        STOP                    (IS,00)
A       DS      1               (DL, 02)  (C,1)
        LTORG                   (DL,05)
        ...                     ...
```

Intermediate code – variant II

```
        START   200             (AD,01)   (C,200)
        READ    A               (IS,09)   A
LOOP    MOVER   AREG, A         (IS,04)   AREG, A
          :                       :
        SUB     AREG, ='1'      (IS,02)   AREG, (L,01)
        BC      GT, LOOP        (IS,07)   GT, LOOP
        STOP                    (IS,00)
A       DS      1               (DL,02)   (C,1)
        LTORG                   (DL,05)
        ...                     ...
```

Memory requirements using variant I (a) and variant II (b)



Back patching

- The **operand** field of an instruction is **containing forward reference is kept blank** initially.
- The address of that symbol is put into field when its **address is encountered.**

Tasks of analysis phase

1. Separate label, opcode & operand

2. Build the symbol table

Analysis

3. Perform LC processing

4. Construct IC

First pass of two pass assembler

- Input
  - Mnemonic opcode table (MOT)
  - Copy of source program (.ASM file)
- Output
  - Location counter
  - OPTAB
  - Symbol table (SYMTAB)
  - Literal table (LITTAB)
  - Pool table (POOL TAB)
  - Intermediate representation
  - Error report

PASS-1 Algorithm

LC	:	Location counter
littab_ptr	:	Points to an entry in LITTAB
pooltab_ptr	:	Points to an entry in POOLTAB

1. LC := 0; (This is the default value)
   littab_ptr := 1;
   pooltab_ptr := 1;
   POOLTAB [1]. *first* := 1; POOLTAB [1]. # *literals* := 0;

2. While the next statement is not an END statement
   (a) If a symbol is present in the label field then
   this_label := symbol in the label field;
   Make an entry (*this_label*, <LC>, –) in SYMTAB.

(b) If an LTORG statement then

(i) If POOLTAB [*pooltab_ptr*]. # *literals* > 0 then

Process the entries LITTAB [POOLTAB [*pooltab_ptr*]. *first*] ... LITTAB [*littab_ptr*−1] to allocate memory to the literal, put address of the allocated memory area in the *address* field of the LITTAB entry, and update the address contained in location counter accordingly.

(ii) *pooltab_ptr* := *pooltab_ptr* + 1;

(iii) POOLTAB [*pooltab_ptr*]. *first* := *littab_ptr*;
POOLTAB [*pooltab_ptr*]. # *literals* := 0;

(c) If a START or ORIGIN statement then
LC := value specified in operand field;

(d) If an EQU statement then

(i) *this_addr* := value of <*address specification*>;

(ii) Correct the SYMTAB entry for *this_label* to (*this_label*, *this_addr*, 1).

(e) If a declaration statement then

(i) Invoke the routine whose id is mentioned in the *mnemonic info* field. This routine returns *code* and *size*.

(ii) If a symbol is present in the label field, correct the symtab entry for *this_label* to (*this_label*, <LC>, *size*).

(iii) LC := LC + *size*;

(iv) Generate intermediate code for the declaration statement.

(f) If an imperative statement then

(i) *code* := machine opcode from the *mnemonic info* field of OPTAB;

(ii) LC := LC + instruction length from the *mnemonic info* field of OPTAB;

(iii) If operand is a literal then

this_literal := literal in operand field;

if POOLTAB [*pooltab_ptr*]. # *literals* = 0 or *this_literal* does not match any literal in the range LITTAB [POOLTAB [*pooltab_ptr*]] .*first* ... LITTAB [*littab_ptr* − 1] then

LITTAB [*littab_ptr*]. *value* := *this_literal*;

POOLTAB [*pooltab_ptr*]. # *literals* :=

POOLTAB [*pooltab_ptr*]. # *literals* +1;

littab_ptr := *littab_ptr* + 1;

else (i.e., operand is a symbol)

this_entry := SYMTAB entry number of operand;

Generate intermediate code for the imperative statement.

3. (Processing of the `END` statement)

   (a) Perform actions (i)–(iii) of Step 2(b).

   (b) Generate intermediate code for the `END` statement.

Steps for implementation of First pass of two pass assembler - 1

1. Create MNEMONIC OPCODE table (OPTAB with mnemonic opcode, class, opcode and length) & make entries in it statically.
2. Create data structures for SYMBOL Table (SYMTAB with symbol, address & length), LITERAL Table (LITTAB with literal & address) and POOL Table (POOLTAB with literal_no) using array of structure.
3. Create and open a source file in read mode having assembly language program (e.g. sample.asm)

Steps for implementation of First pass of two pass assembler - 2

4. Open a target file (sample_IC.txt) in write mode
5. Initialize variables like loc_cntr = 0 & so on
6. Scan/read source file word by word and proceed as per the algorithm i.e. compare tokens with OPTAB.
7. While doing step 6 make entries in SYMTAB, LITTAB, POOLTAB & intermediate code in target file
8. Show errors at the end
9. Close all files

Tasks of synthesis phase

1. Obtain the **machine opcode** corresponding to the mnemonic

2. Obtain the **address of a memory operand** from symbol table

Synthesis

3. **Synthesize** the machine instruction

Algorithm : Second pass of two pass assembler

- Input
  - Machine opcode table(MOT)
  - Location counter
  - Symbol table (SYMTAB)
  - Literal table (LITTAB)
  - Pool table (POOL TAB)
  - Intermediate representation
- Output
  - Machine code
  - Error report

Machine instruction format and example

Sign(1)	Opcode(2)	Reg Operand(1)	Memory Operand(3)

Intermediate code – variant I

```
        START   200             (AD,01)   (C,200)
        READ    A               (IS,09)   (S,01)
LOOP    MOVER   AREG, A         (IS,04)   (1)(S,01)
          :                       :
        SUB     AREG, ='1'      (IS,02)   (1)(L,01)
        BC      GT, LOOP        (IS,07)   (4)(S,02)
        STOP                    (IS,00)
A       DS      1               (DL, 02)  (C,1)
        LTORG                   (DL,05)
        ...                     ...
```

variant I

```
        START   101
        READ            N       101) + 09 0 113
        MOVER   BREG,   ONE     102) + 04 2 115
        MOVEM   BREG,   TERM    103) + 05 2 116
AGAIN   MULT    BREG,   TERM    104) + 03 2 116
        MOVER   CREG,   TERM    105) + 04 3 116
        ADD     CREG,   ONE     106) + 01 3 115
        MOVEM   CREG,   TERM    107) + 05 3 116
        COMP    CREG,   N       108) + 06 3 113
        BC      LE,     AGAIN   109) + 07 2 104
        MOVEM   BREG,   RESULT  110) + 05 2 114
        PRINT   RESUL           111) + 10 0 114
                T
        STOP                    112) + 00 0 000
N       DS      1               113)
RESULT  DS      1               114)
ONE     DC      '1'             115)
TERM    DS      1               116)
        END
```

PASS – 2 of assembler

SYMTAB, LITTAB and POOLTAB

LC	:	Location counter
littab_ptr	:	Points to an entry in LITTAB
pooltab_ptr	:	Points to an entry in POOLTAB
machine_code_buffer	:	Area for constructing code for one statement
code_area	:	Area for assembling the target program
code_area_address	:	Contains address of *code_area*

Algorithm 3.2 (Second pass of a two-pass assembler)

1. *code_area_address* := address of *code_area*;
   pooltab_ptr := 1;
   LC := 0;

2. While the next statement is not an END statement

   (a) Clear *machine_code_buffer*;

   (b) If an LTORG statement

      (i) If POOLTAB [*pooltab_ptr*]. *# literals* > 0 then

         Process literals in the entries LITTAB [POOLTAB [*pooltab_ptr*] .*first*] ... LITTAB [POOLTAB [*pooltab_ptr*+1]−1] similar to processing of constants in a DC statement. It results in assembling the literals in *machine_code_buffer*.

      (ii) *size* := size of memory area required for literals;

      (iii) *pooltab_ptr* := *pooltab_ptr* + 1;

   (c) If a START or ORIGIN statement

      (i) LC := value specified in operand field;

(d) If a declaration statement

   (i) If a DC statement then

      Assemble the constant in *machine_code_buffer*.

   (ii) *size* := size of the memory area required by the declaration statement;

(e) If an imperative statement

   (i) Get address of the operand from its entry in SYMTAB or LITTAB, as the case may be.

   (ii) Assemble the instruction in *machine_code_buffer*.

   (iii) *size* := size of the instruction;

(f) If *size* $\neq$ 0 then

(i) Move contents of *machine_code_buffer* to the memory word with the address *code_area_address* + <LC>;

(ii) LC := LC + *size*;

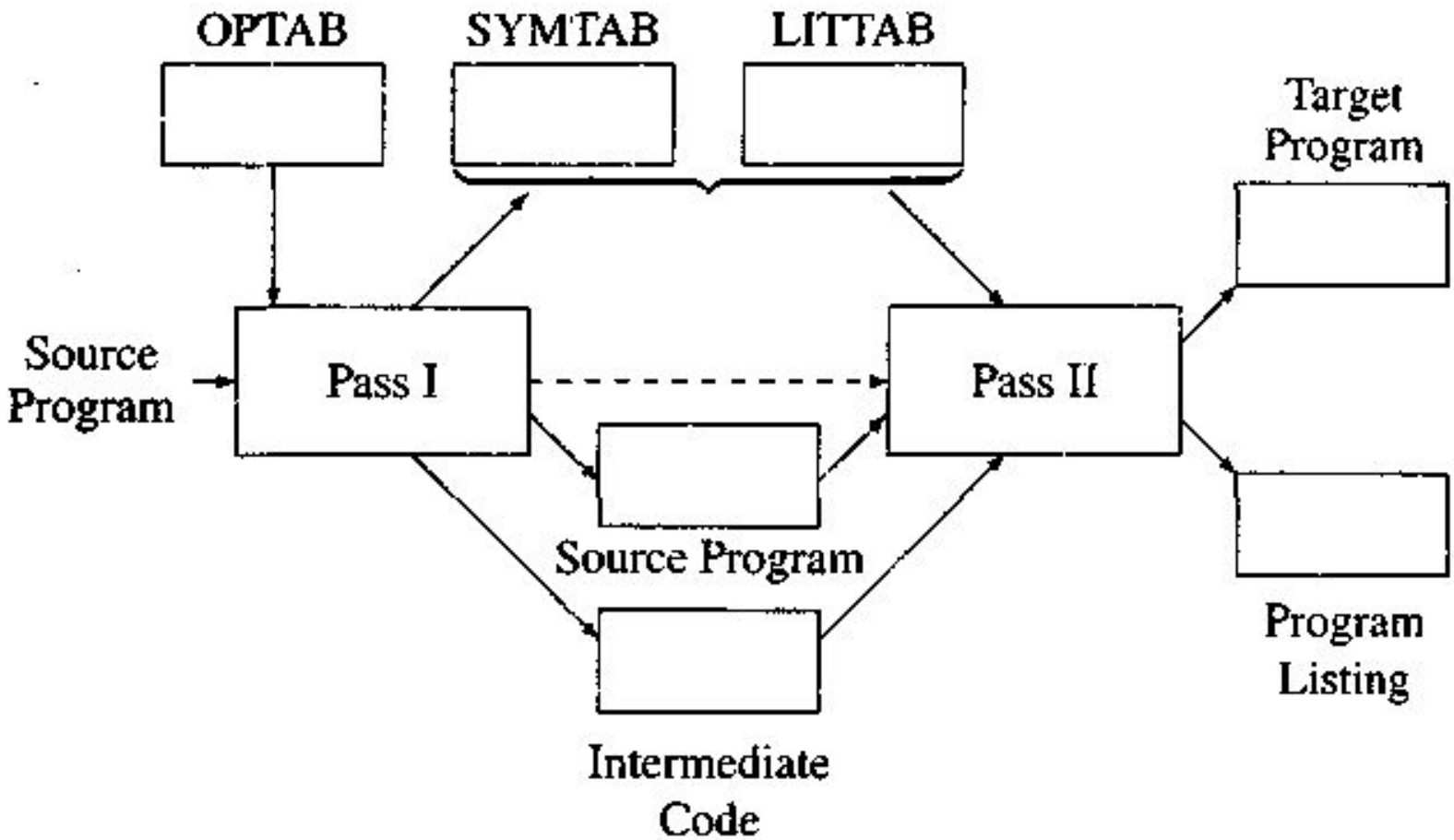
3. (Processing of the END statement)

(a) Perform actions (i)–(iii) of Step 2(b).

(b) Perform actions (i)–(ii) of Step 2(f).

(c) Write *code_area* into the output file.

Data structures and files in a two pass assembler



Error reporting in pass I

```
Sr. No.        Statement                Address

001            START   200
002            MOVER   AREG, A          200
                 :
003
009            MVER    BREG, A          207
          ** error ** Invalid opcode

010            ADD     BREG, B          208
014       A    DS      1                209
                 :
015
021       A    DC      '5'              227
          ** error ** Duplicate definition of symbol A

022              :
035            END
          ** error ** Undefined symbol B in statement 10
```

Error reporting in pass II

```
Sr. No.        Statement                Address   Instruction
001            START   200
002            MOVER   AREG, A          200       + 04 1 209
               :
003
009            MVER    BREG, A          207       + -- 2 209
           ** error ** Invalid opcode
010            ADD     BREG, B          208       + 01 2 ---
           ** error ** Undefined symbol B in operand field
014        A   DS      1                209
               :
015
021        A   DC      '5'              227       + 00 0 005
           ** error ** Duplicate definition of symbol A
               :
022
035            END
```

Pass structures of assemblers

- Two pass translation
- Single pass translation
- Multi pass translation

```
e.g. Alpha  EQU  Beta
     Beta   EQU  Theta
     Theta  EQU  Delta
     Delta  DC   1
```

Summary

- An **assembler** is a program that converts assembly language into machine code.
- Example of assembler
  - MASM, NASM, gas, as86, TASM, a86, Terse, etc.

References

- D. M. Dhamdhere, Systems Programming and Operating Systems, Tata McGraw-Hill, ISBN 13:978-0-07-463579-7, Second Revised Edition
- Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, Compilers Principles, Techniques and Tools, Addison Wesley, ISBN:981–235–885 - 4, Low Price Edition
- J. J. Donovan, Systems Programming, McGraw-Hill, ISBN 13:978-0-07-460482-3, Indian Edition
- https://www.slideshare.net/BhattBalkrishna/ch-3-assembler-in-system-programming

Manisha Mali,
Department of
Computer Engineering,
VIIT , Pune-48

Thank
You!